



HES-SO Master

Master in Engineering - Computer Science

Programmation système Linux et optimisation système Linux

CSEL - Construction de Systèmes Embarqués sur Linux

Auteurs

Carolina Inácio Pereira, Gaby Roch, Simon Mirkovitch

Dépot Git

<https://github.com/g-roch/mse-csel-workspace>



Table des matières

1	Introduction	2
2	Système de fichiers	3
2.1	Résumé du laboratoire	3
2.2	Questions	3
2.3	Ce qui a été appris / exercé	3
2.4	Remarques / À retenir	4
2.5	Feedback personnel	4
2.5.1	Carolina	4
2.5.2	Gaby	4
2.5.3	Simon	4
3	Multiprocessing et Ordonnanceur	5
3.1	Résumé du laboratoire	5
3.2	Questions	5
3.3	Ce qui a été appris / exercé	7
3.4	Remarques / À retenir	8
3.5	Feedback personnel	8
3.5.1	Carolina	8
3.5.2	Gaby	8
3.5.3	Simon	8
4	Outils d'analyse de performance pour Linux	9
4.1	Résumé du laboratoire	9
4.2	Questions	9
4.3	Ce qui a été appris / exercé	11
4.4	Remarques / À retenir	12
4.5	Feedback personnel	12
4.5.1	Carolina	12
4.5.2	Gaby	12
4.5.3	Simon	12
5	Annexes	13
A.	Output de perf stat ./ex1 avant optimisation	13
B.	Output de perf stat ./ex1 après optimisation	13

1 Introduction

Ce rapport présente les travaux réalisés lors des laboratoires consacrés à la programmation système sous Linux embarqué.

La première partie porte sur le contrôle de périphériques matériels via l'interface `sysfs`, en développant une application (en C dans notre cas) permettant de gérer la fréquence de clignotement d'une LED à l'aide de boutons-poussoirs. Elle aborde notamment la gestion des GPIOs via des événements, l'utilisation de `timerfd` pour la génération d'une horloge haute résolution, ainsi que le multiplexage des entrées/sorties.

La deuxième partie traite de la programmation multiprocessus et de la gestion des ressources système, en explorant la communication inter-processus avec `socketpair`, la capture de signaux, l'affinité CPU, ainsi que la mise en œuvre des CGroups pour limiter et contrôler l'utilisation de la mémoire et des cœurs processeur.

La troisième partie introduit les outils d'analyse de performance sous Linux, en particulier `perf`, à travers une série d'exercices pratiques mettant en évidence l'impact du cache L1, du prédicteur de branchement (branch predictor) et des choix algorithmiques sur les performances d'une application.

L'objectif de ce rapport est de résumer les étapes réalisées, de synthétiser les concepts appris et de partager les remarques et retours d'expérience associés à ces laboratoires.

2 Système de fichiers

2.1 Résumé du laboratoire

Ce laboratoire portait sur la manipulation des GPIOs et le contrôle d'une LED via l'interface sysfs de Linux. La partie la plus intéressante était la mise en œuvre du multiplexage des E/S avec select/poll, qui permet d'éviter le busy-waiting identifié dans le code de référence. L'utilisation de timerfd en combinaison avec les événements GPIO offre une architecture propre et efficiente pour gérer simultanément plusieurs sources d'événements.

2.2 Questions

Ce laboratoire ne contenait aucune question.

2.3 Ce qui a été appris / exercé

Le tableau ci-dessous présente les compétences exercées durant les laboratoires selon trois niveaux d'acquisition : *non acquis*, *acquis*, *mais à exercer* et *parfaitement acquis*.

Point / Sujet	Carolina	Gaby	Simon
Analyse d'un code C existant et identification de défauts de performance (busy-waiting)	Acquis, mais à exercer	Acquis, mais à exercer	Parfaitement acquis
Configuration et utilisation des GPIOs via l'interface sysfs	Acquis, mais à exercer	Acquis, mais à exercer	Acquis, mais à exercer
Identification des numéros de GPIO via debugfs	Acquis, mais à exercer	Acquis, mais à exercer	Parfaitement acquis
Configuration des GPIOs en mode événementiel (edge detection)	Acquis, mais à exercer	Acquis, mais à exercer	Acquis, mais à exercer
Utilisation de timerfd pour la génération d'une horloge haute résolution	Acquis, mais à exercer	Acquis, mais à exercer	Acquis, mais à exercer
Multiplexage des entrées/sorties (select/poll/epoll)	Acquis, mais à exercer	Acquis, mais à exercer	Acquis, mais à exercer

Point / Sujet	Carolina	Gaby	Simon
Contrôle de la fréquence de clignotement d'une LED via sysfs	Acquis, mais à exercer	Acquis, mais à exercer	Acquis, mais à exercer
Journalisation des événements avec syslog	Acquis, mais à exercer	Acquis, mais à exercer	Acquis, mais à exercer
Auto-incrémentation/décrémentation de la fréquence par pression continue (optionnel)	N/A	Acquis, mais à exercer	N/A

2.4 Remarques / À retenir

A retenir :

- Le sysfs expose les GPIOs comme de simples fichiers — lire et écrire une valeur revient à lire/écrire dans `/sys/class/gpio/`, ce qui simplifie grandement l'interaction avec le hardware depuis l'espace utilisateur.
- `timerfd` retourne un file descriptor standard, ce qui le rend directement utilisable avec `select/poll/epoll` au même titre que n'importe quel autre fd.
- Le numéro de GPIO ne correspond pas directement au numéro de pin physique — il faut consulter le `debugfs` (`/sys/kernel/debug/gpio`) pour faire la correspondance.

2.5 Feedback personnel

2.5.1 Carolina

Ce laboratoire a été le plus long à réaliser pour moi, en raison de mes difficultés avec les systèmes embarqués en général.

Cela dit, le fait de travailler avec les LEDs et les boutons-poussoirs du NanoPi rend l'exercice concret et ludique, ce qui aide à rester motivée malgré les obstacles. C'est aussi satisfaisant de voir le programme passer de 100% d'utilisation CPU à quasiment 0% grâce au `select` bloquant sur le `timerfd`.

2.5.2 Gaby

Pas de commentaire particulier sur ce laboratoire.

2.5.3 Simon

Je me suis bien amusé durant ce laboratoire à essayer de réduire la consommation du cœur. La gestion des GPIOs m'a permis de réactiver des notions que j'avais perdues et m'a rappelé de bons souvenirs. J'avoue que j'étais un peu intimidé début, car j'ai un peu de peine à bien comprendre la cause de la surconsommation du CPU, mais qui s'est vite dissipé.

3 Multiprocessing et Ordonnanceur

3.1 Résumé du laboratoire

Ce laboratoire couvrait deux aspects distincts de la programmation système : la communication inter-processus et la gestion des ressources via les CGroups. La partie IPC avec socketpair et la gestion des signaux était relativement directe. La partie CGroups était plus exploratoire, notamment la compréhension des interactions entre les sous-systèmes memory et cuset, et la manière dont le noyau réagit lorsque les quotas sont dépassés.

3.2 Questions

Question 1

Quel effet a la commande `echo $$ > /sys/fs/cgroup/memory/mem/tasks` sur les cgroups ?

Cette commande ajoute le processus shell courant (identifié par son PID via `$$`) au groupe de contrôle mémoire spécifié. Les limites définies pour ce cgroup s'appliquent alors à ce processus. Puisque les processus enfants héritent du cgroup de leur parent, le programme lancé depuis ce shell sera lui aussi soumis à la limite mémoire configurée.

Question 2

Quel est le comportement du sous-système memory lorsque le quota de mémoire est épuisé ? Pourrait-on le modifier ? Si oui, comment ?

Lorsque le quota est épuisé, le noyau déclenche l'OOM killer qui tue le processus ayant dépassé la limite, comme on peut l'observer dans nos tests :

```
# echo 5M > /sys/fs/cgroup/memory/mem/memory.limit_in_bytes
# ./build/cgroup_mem_tester
Allocated 1 MiB
...
Allocated 5 MiB
Killed
```

 Bash

Ce comportement peut être modifié via le fichier `memory.oom_control`. En y écrivant 1, l'OOM killer est désactivé : le processus ne sera plus tué, mais recevra des erreurs d'allocation (ENOMEM) lorsqu'il tentera de dépasser la limite.

Question 3

Est-il possible de surveiller/vérifier l'état actuel de la mémoire ? Si oui, comment ?

Oui, en lisant le fichier `memory.usage_in_bytes` du cgroup. Une façon pratique de surveiller l'évolution en temps réel est d'ouvrir un second terminal et d'exécuter :

```
while ;; do cat /sys/fs/cgroup/memory/mem/memory.usage_in_bytes; sleep 1; done
```

 Bash

On peut alors observer l'utilisation croître au fur et à mesure que le programme alloue des blocs, jusqu'à atteindre la limite configurée (ici 25 MiB) :

```
131072
1351680
2400256
...
24551424
25600000
```

 Bash

Question 4

Les 4 dernières lignes de configuration des cpusets sont obligatoires. Pouvez-vous en donner la raison ?

Un cgroup cpuset fraîchement créé hérite de fichiers cpuset.cpus et cpuset.mems vides. Or le sous-système cpuset interdit l'attachement d'une tâche à un cgroup tant qu'il n'a pas au moins un CPU et un nœud mémoire (NUMA) assignés. Un processus doit pouvoir s'exécuter quelque part et allouer de la mémoire quelque part. Sans ces 4 écritures, tout `echo $$ > .../tasks` échouerait avec ENOSPC (« No space left on device »), qui est le code d'erreur historique utilisé par cpuset dans ce cas

Question 5

En plaçant une application dans le cgroup high et une autre dans low, quel devrait être le bon comportement ? Pouvez-vous le vérifier ?

Le shell placé dans high ne peut s'exécuter que sur le CPU 3 ; celui placé dans low que sur le CPU 2. L'application lancée depuis chaque shell hérite de la restriction. Les deux applications tournent en parallèle sur des cœurs différents et ne se concurrencent pas pour le CPU (chacun a 100 % de son cœur). Si les deux cgroups partageaient le même cœur, on verrait par contre la concurrence. Pour le vérifier :

```
ps -o pid,psr,comm -p <pid>          # colonne PSR = CPU courant
taskset -p <pid>                     # affiche l'affinité CPU
cat /proc/<pid>/status | grep Cpus_allowed_list
top                                  # touche 'f' puis activer P (Last Used CPU)
htop                                # afficher la colonne CPU
```

On doit voir le PID du shell « high » toujours sur CPU 3 et celui du shell « low » toujours sur CPU 2.

Question 6

Comment procéder pour lancer deux tâches sur le cœur 4 et attribuer 75% du temps CPU à la première et 25% à la deuxième ?

Il faut d'abord assigner le cœur 4 (index 3) aux deux cgroups :

```
echo 3 > /sys/fs/cgroup/cpuset/high/cpuset.cpus
echo 3 > /sys/fs/cgroup/cpuset/low/cpuset.cpus
```

 Bash

Puis configurer les `cpu.shares` avec un ratio 75/25. La valeur absolue importe peu, seul le ratio compte :

```
echo 75 > /sys/fs/cgroup/cpuset/high/cpu.shares
echo 25 > /sys/fs/cgroup/cpuset/low/cpu.shares
```



On peut vérifier le résultat avec `top` : la somme des processus dans `high` devrait avoisiner 75% et ceux dans `low` 25%. En pratique on observe 74% pour `high` et 24.6% pour `low`, les légers écarts s'expliquant par le fait que les processus héritent des shells parents qui consomment également un peu de CPU.

3.3 Ce qui a été appris / exercé

Point / Sujet	Carolina	Gaby	Simon
Création et gestion de processus enfant avec <code>fork()</code>	Acquis, mais à exercer	Acquis, mais à exercer	Acquis, mais à exercer
Communication inter-processus avec <code>socketpair</code>	Acquis, mais à exercer	Acquis, mais à exercer	Acquis, mais à exercer
Capture et gestion de signaux (SIGHUP, SIGINT, SIGQUIT, SIGABRT, SIGTERM)	Acquis, mais à exercer	Acquis, mais à exercer	Acquis, mais à exercer
Affectation d'un processus à un cœur CPU spécifique (CPU affinity)	Acquis, mais à exercer	Acquis, mais à exercer	Acquis, mais à exercer
Mise en œuvre des CGroups pour limiter l'utilisation mémoire	Acquis, mais à exercer	Acquis, mais à exercer	Acquis, mais à exercer
Mise en œuvre des CGroups pour limiter l'utilisation CPU (<code>cpuset</code>)	Acquis, mais à exercer	Acquis, mais à exercer	Acquis, mais à exercer
Répartition du temps CPU entre cgroups avec <code>cpu.shares</code>	Acquis, mais à exercer	Acquis, mais à exercer	Acquis, mais à exercer
Surveillance de l'état mémoire via les CGroups	Acquis, mais à exercer	Acquis, mais à exercer	Acquis, mais à exercer

3.4 Remarques / À retenir

A retenir :

- Les processus enfants héritent automatiquement du cgroup de leur parent — il suffit donc d'y placer le shell pour que tous les programmes lancés depuis celui-ci soient soumis aux mêmes limites.
- `cpuset.cpus` et `cpuset.mems` doivent être initialisés avant toute affectation de tâche, sinon le noyau refuse l'opération.
- La surveillance de la mémoire en temps réel via `memory.usage_in_bytes` est simple à mettre en place et très utile pour valider le comportement d'un cgroup.

3.5 Feedback personnel

3.5.1 Carolina

J'ai réussi à terminer ce laboratoire avant la fin du cours sans aide extérieure, ce qui était encourageant après les difficultés rencontrées lors des laboratoires précédents.

Les exercices sur les CGroups étaient particulièrement intéressants, car ils permettent de voir concrètement comment le noyau réagit lorsqu'un processus dépasse ses quotas de mémoire ou de CPU.

3.5.2 Gaby

Pas de commentaire particulier sur ce laboratoire.

3.5.3 Simon

Laboratoire très intéressant, car ce sont des notions que j'avais jamais vues ou pas autant détaillées (excepté la concurrence et les threads qu'on voit et revoit dans presque tous les cours).

4 Outils d'analyse de performance pour Linux

4.1 Résumé du laboratoire

Ce laboratoire introduisait `perf`, un outil de profilage puissant mais complexe. Les exercices étaient bien construits et permettaient de découvrir progressivement les différentes sous-commandes. L'exemple du parcours de tableau mettait en évidence de façon très concrète l'impact du cache L1 et du prédicteur de branchement sur les performances. L'optimisation du parseur de logs Apache illustre quant à elle l'importance du choix des structures de données algorithmiques.

4.2 Questions

Question 1

Relevez les compteurs du nombre de context-switches et d'instructions ainsi que le temps d'exécution de `ex1` avec `perf stat`.

Sans options spécifiques, `perf stat` mesure par défaut un certain nombre de compteurs. Voici les résultats obtenus :

- **context-switches** : 22
- **instructions** : 1 669 0330 805
- **temps d'exécution** : 38.53 secondes

Voir l'Annexe A. pour le résultat brut de la commande `perf stat ./ex1` avant optimisation.

Question 2

Ce programme contient une erreur triviale qui empêche une utilisation optimale du cache. De quelle erreur s'agit-il ?

Tel qu'expliqué en cours, il y a deux principes fondamentaux pour que le cache fonctionne bien :

- Localité spatiale
- Localité temporelle

Le programme parcourt un tableau 2D en itérant d'abord sur les colonnes, puis sur les lignes. En C, les tableaux sont stockés en mémoire ligne par ligne (row-major order¹). En parcourant les colonnes en premier, on fait des grands sauts dans la mémoire, ce qui engendre un grand nombre de cache-misses.

Question 3

Corrigez l'erreur, recompilez et mesurez à nouveau le temps d'exécution. Quelle amélioration constatez-vous ?

Pour corriger le problème, il suffit d'inverser les indices `i` et `j` dans la boucle interne, afin de parcourir le tableau ligne par ligne :

¹<https://lucidar.me/en/c-class/lesson-09-04-arrays-and-memory/>

```
array[i][j]++;
```



On passe de 38.5 secondes à 2.5 secondes, soit une amélioration d'un facteur 15.

Voir Annexe B. pour le résultat brut de la commande `perf stat ./ex1` après optimisation.

Question 4

Relevez les valeurs du compteur `L1-dcache-load-misses` pour les deux versions. Quel facteur constatez-vous entre les deux valeurs ?

- **Version originale** : 407'185'523 `L1-dcache-load-misses`
- **Version corrigée** : 1'305'355 `L1-dcache-load-misses`

On constate un facteur d'environ 312 ($\frac{407185523}{1305355} \approx 312$), ce qui confirme que l'erreur de parcours avait un impact massif sur l'utilisation du cache.

Question 5

Décrivez brièvement ce que sont les évènements suivants : instructions, cache-misses, branch-misses, L1-dcache-load-misses, cpu-migrations, context-switches.

- **instructions** : nombre total d'instructions machine exécutées par le processeur.
- **cache-misses** : nombre d'accès mémoire n'ayant pas trouvé la donnée dans le cache (dernier niveau).
- **branch-misses** : nombre de fois où le prédicteur de branchement s'est trompé, forçant un vidage du pipeline.
- **L1-dcache-load-misses** : nombre d'accès en lecture n'ayant pas trouvé la donnée dans le cache de données L1.
- **cpu-migrations** : nombre de fois où un processus a été déplacé d'un cœur CPU à un autre par l'ordonnanceur.
- **context-switches** : nombre de commutations de contexte, c'est-à-dire de changements de processus actif sur un cœur.

Question 6

Mesurez le temps d'exécution de `ex1` avec et sans `perf stat`. Quel est l'impact sur les performances ?

- **Avec perf stat** : 2.64 secondes (real)
- **Sans perf stat** : 2.45 secondes (real)

L'overhead introduit par `perf stat` est très faible (environ 8% dans ce cas), ce qui confirme que `perf` est un outil de profilage à faible impact sur les performances.

Question 7

Décrivez en quelques mots ce que fait le programme `ex2`.

Le programme génère un tableau de valeurs aléatoires de type `short`, puis itère sur ce tableau en accumulant dans une somme uniquement les valeurs positives. Il mesure le temps nécessaire à cette opération.

Question 8

Compilez et mesurez le temps d'exécution de la version modifiée de ex2 (avec qsort). Quelle amélioration constatez-vous ?

- **Version originale** : 26.17 secondes
- **Version triée** : 23.43 secondes

L'amélioration est modeste (3 secondes, soit environ 10%). Contrairement à ce qu'on pourrait attendre, le tri n'apporte pas une amélioration spectaculaire sur cette plateforme.

Question 9

À l'aide de perf stat, déterminez pourquoi le programme modifié de ex2 s'exécute plus rapidement.

Même si le gain en temps d'exécution reste modeste, les compteurs perf révèlent une différence frappante sur les branch-misses :

- **Version originale** : 327'853'090 branch-misses (33.16% des branches)
- **Version triée** : 811'691 branch-misses (0.08% des branches)

Une fois le tableau trié, la condition `if (data[i] >= 0)` présente un motif régulier que le prédicteur de branchement anticipe correctement dans presque tous les cas. Le gain en temps d'exécution reste cependant limité, ce qui suggère que d'autres facteurs (accès mémoire, latence) dominant sur cette architecture.

4.3 Ce qui a été appris / exercé

Point / Sujet	Carolina	Gaby	Simon
Utilisation de perf stat pour mesurer les compteurs de performance	Acquis, mais à exercer	Acquis, mais à exercer	Acquis, mais à exercer
Identification et correction d'erreurs de cache (cache-misses)	Acquis, mais à exercer	Acquis, mais à exercer	Acquis, mais à exercer
Analyse de l'impact du prédicteur de branchement (branch-misses)	Acquis, mais à exercer	Acquis, mais à exercer	Acquis, mais à exercer
Profilage d'une application avec perf record et perf report	Acquis, mais à exercer	Acquis, mais à exercer	Acquis, mais à exercer

Point / Sujet	Carolina	Gaby	Simon
Identification des fonctions critiques via le call graph	Acquis, mais à exercer	Acquis, mais à exercer	Acquis, mais à exercer
Optimisation algorithmique guidée par les résultats de perf	Acquis, mais à exercer	Acquis, mais à exercer	Acquis, mais à exercer
Mesure de la latence et de la gigue d'interruption (kernel space et user space)	Acquis, mais à exercer	Acquis, mais à exercer	Acquis, mais à exercer

4.4 Remarques / À retenir

Pas de remarques particulières

4.5 Feedback personnel

4.5.1 Carolina

J'apprécie que ce laboratoire ait été plus léger, étant donné que la charge de travail à la fin du semestre est plus élevée. Les exercices étaient bien guidés et les résultats immédiatement visibles, ce qui rendait la progression agréable.

La démonstration de l'impact du parcours de tableau sur le cache était particulièrement intéressante, avec un facteur 312 sur les cache-misses pour un simple échange d'indices.

J'ai également découvert les principes de localité spatiale et temporelle qui gouvernent l'efficacité du cache, ainsi que la façon dont C organise la mémoire.

4.5.2 Gaby

Pas de commentaire particulier sur ce laboratoire.

4.5.3 Simon

Labo léger qui fait du bien durant cette fin de semestre. Il reste intéressant et amusant de voir la différence de performance entre avant et après les optimisations. J'aurais bien aimé de tester les différentes options du compilateur et voir ce qu'elles changent en terme de performance ainsi que si le code reste le même ou pas.

5 Annexes

A. Output de perf stat ./ex1 avant optimisation

Performance counter stats for './ex1':

38517.08 msec	task-clock	#	1.000 CPUs utilized
22	context-switches	#	0.571 /sec
0	cpu-migrations	#	0.000 /sec
48866	page-faults	#	1.269 K/sec
31429594335	cycles	#	0.816 GHz
1669033805	instructions	#	0.05 insn per cycle
269367271	branches	#	6.993 M/sec
1008741	branch-misses	#	0.37% of all branches

38.526592144 seconds time elapsed

37.799067000 seconds user

0.335553000 seconds sys

B. Output de perf stat ./ex1 après optimisation

Performance counter stats for './ex1':

2458.91 msec	task-clock	#	0.996 CPUs utilized
17	context-switches	#	6.914 /sec
0	cpu-migrations	#	0.000 /sec
48867	page-faults	#	19.873 K/sec
2006326563	cycles	#	0.816 GHz
1380397510	instructions	#	0.69 insn per cycle
264956230	branches	#	107.754 M/sec
647655	branch-misses	#	0.24% of all branches

2.468533084 seconds time elapsed

2.195271000 seconds user

0.241887000 seconds sys